BEAST 3 Package Manager

Design, Storage Models, and Loading Precedence

BEAST Core Developer Team

2026

Package System Design Principles

1. **One ModuleLayer per package** — namespace isolation without losing visibility to the core API. Two packages can bundle different versions of the same library without conflict.
2. **Same code path for dev and deployment** — `module-info.java` `provides` declarations are the single source of truth. A developer in IntelliJ and an end user with an installed ZIP see the same service discovery.
3. **Graceful degradation** — modules with unsatisfied dependencies (e.g. FX modules on a headless cluster) are silently skipped. Core modules still load.
4. **First-found wins** — when the same JPMS module appears in multiple locations (boot layer, installed package, archive), the first one loaded wins. No silent duplicates, no "reads more than one module" errors.
5. **Dual distribution** — ZIP archives (CBAN) and Maven Central JARs converge on the same ModuleLayer creation path.

Development vs Deployment

Development (IDE)

- All modules in the **boot layer**
- IDE resolves module path from Maven
- Package manager has nothing to do
- Services discovered via `module-info.java`
- Debug, hot-reload as normal

Deployment (user install)

- Only core modules in boot layer
- External packages loaded into child **ModuleLayers** at startup
- Two storage formats coexist
- Same service discovery mechanism

Service Discovery: `module-info.java`

Primary mechanism — BEAST scans module descriptors at startup:

```
open module my.beast.package {
    requires beast.pkgmgmt;
    requires beast.base;

    provides beast.base.core.BEASTInterface with
        my.beast.package.MyModel,
        my.beast.package.MyOperator;
}
```

- Works in both the boot layer (IDE development) and plugin layers (deployed packages)
- IntelliJ/Eclipse discover services automatically when both projects are open

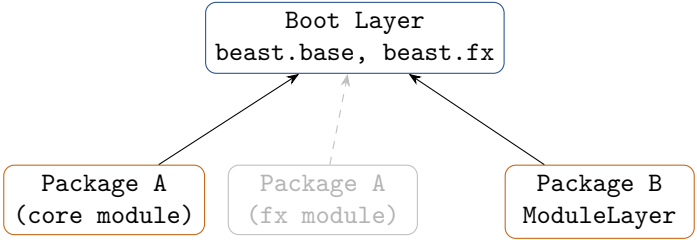
Service Discovery: `version.xml` Fallback

- **Still works** for non-modular JARs
- JARs without `module-info` are treated as automatic modules
- Services registered from `version.xml` as before
- Useful for gradual migration

Recommendation

Add `module-info.java` when you can — it's the primary mechanism and gives you compile-time module dependency checking.

Plugin ModuleLayer Isolation



- Each deployed package gets its **own ModuleLayer**
- No classpath conflicts between packages
- Modules with unsatisfied dependencies (e.g. FX modules on a headless cluster) are **automatically skipped** — core modules still load

Two Storage Models

Both formats live under `~/.beast/2.8/`:

ZIP packages (legacy)

- One **subdirectory per package**:
  `MyPackage/lib/*.jar`
  `MyPackage/version.xml`
- Directory structure *is* the index — any subdirectory with `version.xml` is a package
- No external index needed

Maven packages (recommended)

- `maven-packages.xml` — which coordinates the user installed
- `maven-repo/` — local JAR cache (package + transitive deps)
- Remote repos: Maven Central + extras in `beauti.properties`
- Index needed because the cache is opaque — can't tell "what is a package" from directory structure alone

Why two models?

ZIP packages are self-describing (directory = package). Maven packages need an explicit index because the local cache holds transitive dependencies alongside the package itself.

Package Loading Precedence

`PackageManager.loadExternalJars()` runs when BEAST starts:

1. **Load Maven packages** — resolve coordinates from `maven-packages.xml`, create a ModuleLayer per package. Maven is the recommended format, so these take precedence.
2. **Scan package directories** — find installed ZIP packages in `~/.beast/2.8/` (or `BEAST_PACKAGE_PATH`)
3. **Create ModuleLayer per ZIP package:**
   - Discover JPMS modules from the package's JARs
   - Exclude modules already in boot layer or Maven layers (no duplicates)
   - Filter out modules whose `requires` can't be satisfied
   - `resolveAndBind()` on the remaining modules
4. **Register with `BEASTClassLoader`** — services from both `module-info.java` and `version.xml` merged into registry

Precedence: boot layer → Maven → ZIP. If the same module exists in both formats, the Maven version wins.

Maven Central Distribution

New: packages can be published as plain Maven Central JARs.

- Users install via BEAUti: **Install from Maven** button
- Or CLI: `packagemanager -maven groupId:artifactId:version`
- Resolution via Apache Maven Resolver

Three components:

- `maven-packages.xml` — records installed coordinates
- `~/.beast/2.8/maven-repo/` — local JAR cache
- Remote repos: Maven Central (always) + custom repos stored in `beauti.properties`

- `packagemanager -addMavenRepository <url>` — add a custom repo
- ZIP packages still work alongside Maven packages

beast-package-skeleton

Ready-to-copy template project on GitHub:

- Maven `pom.xml` with BEAST 3 dependencies
- `module-info.java` with `provides` declarations
- Example `ScalarDistribution`, operator, and tests
- BEAST XML using the strongly-typed spec API
- Maven Central release profile (source, javadoc, GPG, publishing plugin)
- `version.xml` JAR embedding

Start here: `https://github.com/CompEvol/beast-package-skeleton`